

```

In [ ]: # Import packages
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

import pennylane as qml
from pennylane import numpy as qnp

# Load the dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/forest-fires/forest-fires.csv"
df = pd.read_csv(url)

# Create binary target: fire occurred or not
df['fire'] = (df['area'] > 0).astype(int)

# Encode categorical columns
df['month'] = LabelEncoder().fit_transform(df['month'])
df['day'] = LabelEncoder().fit_transform(df['day'])

# Prepare features and target
X = df.drop(columns=['area', 'fire'])
y = df['fire'].values

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert labels to -1 and 1 for compatibility with quantum outputs
y_mapped = 2 * y - 1

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_mapped, test_size=0.2, random_state=42)

# Quantum circuit setup
n_qubits = X_train.shape[1]
dev = qml.device("default.qubit", wires=n_qubits)

def angle_encoding(x):
    for i in range(n_qubits):
        qml.RY(x[i], wires=i)

def variational_circuit(weights, x=None):
    angle_encoding(x)
    qml.templates.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0)) # Scalar output

@qml.qnode(dev, interface="autograd")
def circuit(weights, x):
    return variational_circuit(weights, x)

# Cost function (MSE between circuit output and label)
def cost(weights, X, y):
    losses = []
    for i in range(len(X)):
        output = circuit(weights, X[i])
        loss = (output - y[i]) ** 2
        losses.append(loss)

```

```

    return qnp.mean(qnp.stack(losses))

# Initialize weights
layers = 4
weights = qnp.random.normal(0, qnp.pi, (layers, n_qubits, 3), requires_grad=True)

# Training Loop
opt = qml.AdamOptimizer(stepsize=0.1)
epochs = 10

for epoch in range(epochs):
    weights, loss = opt.step_and_cost(lambda w: cost(w, X_train, y_train), weights)
    if epoch % 5 == 0:
        print(f"Epoch {epoch}, Loss = {loss:.4f}")

# Prediction function (thresholded at 0)
def vqc_predict(weights, x):
    return 1 if circuit(weights, x) > 0 else 0

# Run predictions
y_pred = [vqc_predict(weights, x) for x in X_test]

# Map test labels back to 0 and 1
y_test_binary = (y_test + 1) // 2

# Evaluation
print("\nClassification Report:")
print(classification_report(y_test_binary, y_pred))
print("Accuracy:", accuracy_score(y_test_binary, y_pred))
print("Precision:", precision_score(y_test_binary, y_pred))
print("Recall:", recall_score(y_test_binary, y_pred))
print("F1 Score:", f1_score(y_test_binary, y_pred))

```

Epoch 0, Loss = 1.0012

Epoch 5, Loss = 0.9847